

Python as a First Programming Language for Everyone

Dr. Toby Donaldson
Assistant Professor
Simon Fraser University
tjd@sfu.ca

Abstract

In this paper, the Python programming language is introduced, and promoted as an excellent choice as a first programming language for anyone who wants to learn programming. We give a brief history and synopsis of what makes it so popular, followed by a series of code examples comparing it to Java. Finally, it is argued that the rise in popularity of the World Wide Web has drastically changed the nature of programming, and that Python is a better general-purpose introductory language than Java, C++, or other traditional high-level languages.

1. A Brief History of Python

Python is a dynamically-typed programming language that supports object-oriented and functional programming.^[1] It's the brainchild of Guido van Rossum, who began writing Python in late 1989 and early 1990 as the scripting language for the Amoeba operating system.^[2] Its design was inspired by a number of other languages, including C, Modula-3, and particularly the educational language ABC.^[3] Van Rossum released it publicly in 1991, and since then Python has been gathering a large and enthusiastic group of users including professional programmers, educators, and computer scientists. Van Rossum, who is known to all as just Guido, or sometimes the BDFL ("benevolent dictator for life"), is still Python's lead developer and designer. Python gets its name from the Monty Python comedy troupe, and Python culture is peppered with references to those irreverent comedians (e.g. IDLE, after Eric Idle, is the name of the IDE that comes with Python).

2. The Flavour

Python is an elegant, simple, and practical language. It began life as a scripting language, and while that is still probably its most common application, it has been used to solve many other problems. It compares favourably to Perl, the most obvious difference being Python's clean, easy to read syntax; Python programs are typically exceptionally readable and understandable. In contrast, Perl is often disparaged for its "line noise" syntax, due to its many arbitrarily chosen operators. Like Perl, Python is run as a large and successful open source project.^[4] It uses a collaborative design process where anyone can offer a new feature written up as a "Python Enhancement Proposal", or PEP.^[5] PEPs are debated on the comp.lang.python newsgroup, or on one of the developer mailing lists. The final decision on which PEPs to accept is made either by developer consensus, or through a "pronouncement" by Guido.

There is not one single feature of Python that makes it such a good language. It has cherry-picked many of the best features from other languages, and combined them in a clear and synergistic way. One of Python's slogans is "batteries included". Python can do many useful things right out of the box, and provides all the basic tools and features that programmers have come to expect of a major programming language. It comes with a graphical IDE (including keyword coloring editor, a debugger, and an interactive command shell --- all written in Python with source code included), the Tkinter GUI library, a profiler, and numerous other tools that most other languages require you to add as extra packages. Not all these tools do everything a professional might need, but they often do; for beginning programmers, they are generally quick to learn and easy to use.

Like C++, Python is a multi-paradigm language that does not force programmers to follow a single pure style of programming. Python can be used as an imperative language, an object-oriented language, or a functional language. In practice, the three styles are often intermixed in complementary ways, allowing the programmer to use the best tool for the job at hand.

3. The Three Faces of Python

Python is perhaps first programming language to have hit a particular programming sweet spot: it appeals to beginner's, professionals, and computer scientists alike.

Beginners just learning to program appreciate Python's simplicity, interactivity, and "batteries included" flavour. They can quickly get Python to start doing interesting things by using its intuitive syntax and data structures. Thanks to the interactive command line, they get plenty of immediate feedback, and can study individual commands in isolation, without worry about full-scale programs. There is no code-compile-run cycle, and, initially, there is no need to use files or classes or other overhead that commonly cause beginners unnecessary headaches. Within the Python community itself, the pedagogical value of Python is recognized and promoted.^[6] Most of the rest of this paper will be arguing for the use of Python in a CS curriculum at the university level.

Many professional programmers are turning to Python, often as an alternative to Perl, or other scripting languages. Python has good versions of most of the essential code libraries needed in practical day-to-day programming.^[7] Like Perl, Python is excellent for scripting, and string manipulation, yet its syntax is much less cryptic. Python can be learned quickly, and for many experienced programmers the initial investment of effort into learning enough about Python to use it practically is remarkably small. Python has been promoted by programming luminaries such as open source celebrity Eric Raymond,^[8] and Google's head of search quality, Peter Norvig.^[9] It has also been used by companies ^[10] such as IBM, Disney, NASA, Industrial Light and Magic, and Zope Corporation.^[note 1]

Finally, even computer scientists have much to like about Python. Peter Norvig describes Python as being "either a practical (better libraries) version of Scheme, or as a cleaned-up (no \$@&%) version of Perl".^[11] Python mixes functional programming ideas such as higher-order functions, list comprehensions, simple generators, iterators, and objects. It can be used to illustrate a number of different programming styles, although within one simple and consistent language. Thanks to Python's open source, Python has been used as the base for other languages, such as Stackless.^[12]

4. A Comparison to Java

Python and Java make for an interesting comparison, especially when considering the concepts and ideas they introduce in a first

programming course. In the following section, a number of Python features and issues will be raised, and compared to Java with specific code examples. This is far from an exhaustive comparison, and it is admittedly somewhat biased to showing the best features of Python.

4.1 Hello, World

Perhaps the single biggest difference between Python and Java, for beginners, is that Python is an interpreted language that allows almost any expression to be evaluated at the command line. Java is of course a compiled language, with the familiar code-compile-run cycle. The problem with this cycle is that it makes running programs a slower and more complex process, ultimately degrading the quality of feedback. The "Hello, world" program in Python is about as simple as it can get:

```
>>> print "Hello, world!"
Hello, world!
```

The ">>>" is Python's input prompt, and on the following line is the output. The program can also be written in a file to be run as a script. It is straightforward to run the command-line interpreter, or, even better, the IDLE integrated environment that comes packaged with Python. Beginners don't need to be concerned about files, or compiling. They can just type in the code and look at the results.

In Java, much more overhead is needed:

```
class HelloWorld {
    public static void main(String[] args)
    {
        System.out.println("Hello, world!");
    }
}
```

The problems with this Java code are easy to enumerate. It requires a perplexing number of concepts superfluous to the problem of printing a message on the screen, e.g. classes, statics, void, arrays, and parameter passing. To run it, two more steps are required. It must first be explicitly compiled, and then run on the Java virtual machine. Of course, if you are using the javac compiler that comes with the standard JDK, then to avoid a compiler error you must also ensure that the code is stored in a file named HelloWorld.java. A good IDE [\[note 2\]](#) can make this a simpler process, but learning one of these IDEs, except perhaps for BlueJ, introduces new complexities and issues beyond the language itself (e.g. project files, configuration options). The problem is that Java is a language designed for professional programmers whose daily work consists largely in the creation, extension, and maintenance of large and complex software systems. Little consideration was given to beginners in the design of Java, beyond trying to make it easier to use than C++. While Java may indeed be easier to program in than C++, that fact alone does not mean it is the best language for introductory programming.

It is worth saying a few more words about the BlueJ IDE, which was explicitly designed for teaching Java. [\[13\]](#) The BlueJ home site lists a number of common Java problems and complexities that BlueJ hides. BlueJ provides a debugger, and a graphical design tool that can be used to structure and relate classes, and even run portions of Java code not part of an entire program. What BlueJ does is make Java simpler and more interactive for beginners, features that are already integral to Python. Another issue with BlueJ is that it is coupled with an aggressive "object's first" approach to programming, emphasizing data over algorithms. While "object's first" is a popular and reasonable approach to teaching Java, not all educators choose to teach object-oriented design as early as others do, and so many of the benefits of BlueJ are not realized in, say, an algorithms-first curriculum.

4.2 Whitespace

One of the most unusual features of Python, at least for experienced programmers, is Python's use of indentation to indicate blocks of code. Java uses "{" and "}" to explicitly mark the begin and end of code blocks, but Python relies on whitespace. The major advantage is that the compiler helps to enforce proper indentation, making it harder than normal to write poorly indented code. In other languages, indentation is an art that students must learn with no little or no automation. In Python, it's an intrinsic part of the language. As an example, here is a Python version of linear search:

```
def search(x, seq):
    """ Returns true if x is seq, and false
    otherwise.
    """
    for item in seq:
        if x == item:
            return 1    # 1 means true
    return 0    # 0 means false
```

If, for instance, the "for" line was moved a few spaces to the left, then a syntax error would result. Note also that a ":" is used to indicate the start of a code block. This comes from experience with the ABC language, which indicates that the ":" makes it easier to recognize blocks. The triple-quotes "" at the top are a convenient way of writing multi-line strings in Python, and are commonly used for documentation strings.

4.3 Built-in Data Structures

In addition to basic numeric types, Python provides three ubiquitous built-in data structures: strings, list, and dictionaries. Strings and lists are simple and common enough to be introduced on the first or second day of a Python programming course. They support a flexible slicing notation that can extract any substring or sublist. For example:

```
>>> lst = [1, "cat", 4, "a", "dog"]
>>> lst[0]
1
>>> lst[2:4]
[4, 'a']
>>> lst[2:]
[4, 'a', 'dog']
>>> lst[:4]
[1, 'cat', 4, 'a']
```

```
[1, 'cat', 4, 'a']
>>> lst[-1]
'dog'
>>> lst[:-2]
[1, 'cat', 4]
```

Note the negative indices in the last two examples. They refer to items at the right end of a list, and, while unusual at first, this turns out to be a handy notation. Strings are indexed in the same way, and in fact any class sufficiently like a sequence can use the same slicing notation.

Java can do similar manipulations using a `Vector`, or any class implementing the `List` interface. For instance:

```
import java.util.Vector;

public class ListManipulation {

    public static void main(String[] args) {
        Vector lst = new Vector();
        lst.add(new Integer(1));
        lst.add("cat");
        lst.add(new Integer(4));
        lst.add("a");
        lst.add("dog");

        System.out.println(lst.get(0));
        System.out.println(lst.subList(2, 4));
        System.out.println(lst.subList(2, lst.size()));
        System.out.println(lst.subList(0, 4));
        System.out.println(lst.get(lst.size() - 1));
        System.out.println(lst.subList(0, lst.size() - 2));
    }
} // ListManipulation
```

The output:

```
1
[4, a]
[4, a, dog]
[1, cat, 4, a]
dog
[1, cat, 4]
```

Needless to say, Java's approach is wordier, and less elegant than Python's.

Python dictionaries are only slightly more complicated than lists. A dictionary is a set of key/value pairs stored as a hash table. Like lists, they have their own literal notation, e.g.

```
>>> dict = {1: "car", "nose": "hat", 34: [1, 2, 3]}
```

creates a dictionary where, for instance, 1 is the key for the value "car". Accessing a value is straightforward:

```
>>> dict["nose"]
'hat'
```

Java has a few choices for implementing dictionaries. For instance, here is an essentially equivalent program using `HashMap`:

```
import java.util.HashMap;
import java.util.Vector;

public class Dictionary {

    public static void main(String[] args) {
        HashMap dict = new HashMap();
        dict.put(new Integer(1), "car");

        dict.put("nose", "hat");

        Vector lst = new Vector();
        lst.add(new Integer(1));
        lst.add(new Integer(2));
        lst.add(new Integer(3));
        dict.put(new Integer(34), lst);

        System.out.println("dict = " + dict);
    }
} // Dictionary
```

The following output is produced:

```
dict = {34=[1, 2, 3], 1=car, nose=hat}
```

In general, Python gives special privileges to strings, list, and dictionaries, while Java only privileges strings. In fact, Java under privileges primitive types like ints and doubles, forcing the programmer to use ugly and inconvenient wrapper objects.[\[note 3\]](#) Since lists and dictionaries are built-in types, Python provides a useful literal notation for them, which greatly simplifies creating, understanding, and using them. When producing the above examples, I was about equally familiar with Java and Python, having more experience with Java. Yet, overall the Java examples probably took 5 to 10 times longer to produce than the Python examples. Reading the documentation, creating the code, and the time spent dealing with small errors (e.g. writing `HashMap` instead of `HashMap`) adds up.

4.4 Mathematical Synergy

While Python is not as mathematically suave as, say, Maple or Matlab, it behaves more like regular mathematics than Java in ways that beginners will appreciate. For example, in Java, integers are finite 32 bit values with a minimum and maximum value. Beginning programmers are often surprised to learn that Java's arithmetic is not quite the same as the arithmetic they were taught in elementary school. The following Java code will eventually stop because the incremented variable eventually hits the maximum int and then wraps around to a negative value:

```
int i = 1;
while (i > 0) {    // Java: eventually halts
    ++i;
}
```

This particular code is just a curiosity, but finite-valued ints are a real limit that programmers must always keep in mind. Python, however, uses arbitrary precision integers by default (32-bit values are used when the numbers are small enough). That means that the Python equivalent of the above code will loop forever (or until your computer runs out of time or memory):

```
i = 0
while i > 0:    # Python: runs forever
    i += 1
```

More practically, this means that beginners don't need to initially worry about hitting the floor or ceiling of integers. Integer arithmetic works the way they were taught in school. Of course, all programmers should eventually understand how to use finite arithmetic, but Python lets you initially ignore this.

Real-value floating point numbers in Python work essentially the same as in Java. However, Python provides built-in support for complex arithmetic. For instance, to calculate the square root of -1 you can use the complex math square root function:

```
>>> import cmath
>>> cmath.sqrt(-1)
1j
```

The suffix "j" represents the square root of -1, and can be used in numeric expressions. As an example, we can calculate the roots of a quadratic equation using the following function:

```
def quadratic(a, b, c):
    """ Returns both solutions of the quadratic
    equation ax^2 + bx + c = 0.
    """
    import cmath
    sol1 = (-b + cmath.sqrt(b ** 2 - 4 * a * c)) / 2
    sol2 = (-b - cmath.sqrt(b ** 2 - 4 * a * c)) / 2
    return sol1, sol2
```

The ** operator means exponentiation. Here are some sample runs:

```
>>> quadratic(1,2,3)
((-2+2.8284271247461903j), (-2-2.8284271247461903j))

>>> quadratic(1,-3,2)
((4+0j), (2+0j))
```

The outer brackets of the results are because the quadratic function returns its values as a tuple, and complex numbers are always written with surrounding brackets. Another small but nice feature of Python is that it interprets inequalities like "a < b < c" in the expected way. An expression like "a < b < c" is illegal in Java,[\[note 4\]](#) forcing you to translate it as

```
if (a < b && b < c) {
    // do something
}
```

In Python, no such translation is needed:

```
if a < b < c:
    # do something
```

Sometimes this notation can make algorithms much clearer than their Java counterpart. For instance, suppose you have four integer variables that you want to sort into ascending order, with the added restriction that you may not use an array or container.[\[note 5\]](#) We can write a version of bubblesort to solve this problem, which is a nice exercise that mixes loops, if-statements, and variables. Here is how it can be done in Python:

```
def four_sort(a, b, c, d):
    while not a < b < c < d:
        if a > b:
            a, b = b, a
        elif b > c:
            b, c = c, b
        elif c > d:
            c, d = d, c
    return a, b, c, d
```

Note that Python supports multiple assignment. An expression such as "x, y = 1,2" assigns 1 to x and 2 to y, in parallel. You can then swap two variables by writing "x,y = y,x". The function outputs the given values as a tuple:

```
>>> four_sort(4,3,2,1)
(1, 2, 3, 4)
```

The Java equivalent is trickier because Java has no easy way to swap primitive variables, and methods cannot return multiple values. A reasonable equivalent is this:

```
class Foursort {
```

```

private static int a, b, c, d;

public static void fourSort() {
    while (! (a < b && b < c && c < d)) {
        if (a > b) {
            int temp = a;
            a = b;
            b = temp;
        } else if (b > c) {
            int temp = b;
            b = c;
            c = temp;
        } else if (c > d) {
            int temp = c;
            c = d;
            d = temp;
        } // if
    } // while
}

public static void main (String[] args) {
    a = 4;
    b = 3;
    c = 2;
    d = 1;
    fourSort();
    System.out.println("a = " + a + ", b = " + b + ", c = " + c + ", d = " + d);
} // main
}

```

Again, the Python solution has much less overhead, and lets the programmer more clearly focus on the task they want to achieve, instead of on the details of the programming language.

More recently (Python 2.3 and beyond), Python has added a standard set data structure. Python sets represent finite unordered collections of distinct objects, and support all the familiar set operations. Python allows operator overloading, so if A and B are sets, then, for example, $A \cup B$ is their union, and $A \subseteq B$ is true if and only if A is a subset of B. The educational value of sets is quite high given their prominence in mathematics, and finite sets are extremely common in discrete mathematics. Python's list comprehension notation is also a natural way to initialize sets. For example, suppose we want to represent the set of the squares of the numbers from 1 to 10. Mathematically, we can write this set in a couple of ways, e.g.

$$S = \{x^2 : 1 \leq x \leq 5\}$$

or

$$S = \{x^2 : x = 1, 2, 3, 4, 5\}$$

In Python, we can write this:

```

>>> Set([x**2 for x in [1,2,3,4,5]])
Set([16, 1, 4, 25, 9])

```

The expression "[x**2 for x in [1,2,3,4,5]]" is known as a list comprehension, and is a convenient shorthand for creating lists. [\[note 6\]](#) The displayed version of the set is not in the same order as the input version because sets are implemented on top of Python's dictionary data structure, which are in turn implemented as hash tables, and so no particular element ordering is guaranteed. Python's list data type is used for ordered sequences.

Java's standard collections library provides a couple of different set classes. For instance, here is how a HashSet can be used to calculate S:

```

public class SquareSet {

    public static void main(String[] args) {
        HashSet S = new HashSet();
        for (int i = 1; i <= 5 ; ++i) {
            S.add(new Integer(i * i));
        } // for

        System.out.println("S = " + S);
    }

} // SquareSet

```

The output is:

```
S = [4, 9, 16, 1, 25]
```

Using Java collections is not terribly difficult, and within the grasp of beginning programmers. However, like much of Java, there is a lot of overhead, both necessary and unnecessary, and the notation is so different that the analogy with mathematics is weak at best.

4.5 Documentation

Documentation is another win for Python. Java is famous for its reams of on-line documentation, most of it created directly from source code using the javadoc tool. However, for beginners, the Java documentation can be a mass of confusing technical arcana. It takes time to learn how to use and read the Java documentation, to know what's important, and what's not. Even for expert programmers, it's always a bit of an interruption to have to go look up the methods and behaviour of a particular class. In contrast, most of the Python standard library and built-in functions come with their own documentation. For instance, to get a list of all the string methods of a string, one uses the dir command like this:

```
>>> dir("")
```

```
[ '_add_', '_class_', '_contains_', '_delattr_', '_doc_',
  '_eq_', '_ge_', '_getattribute_', '_getitem_', '_getslice_',
  '_gt_', '_hash_', '_init_', '_le_', '_len_', '_lt_',
  '_mul_', '_ne_', '_new_', '_reduce_', '_repr_', '_rmul_',
  '_setattr_', '_str_', 'capitalize', 'center', 'count', 'decode',
  'encode', 'endswith', 'expandtabs', 'find', 'index', 'isalnum',
  'isalpha', 'isdigit', 'islower', 'isspace', 'istitle', 'isupper',
  'join', 'ljust', 'lower', 'lstrip', 'replace', 'rfind', 'rindex',
  'rjust', 'rstrip', 'split', 'splitlines', 'startswith', 'strip',
  'swapcase', 'title', 'translate', 'upper']
```

Admittedly, this output is initially messy and scary for beginners ("what are all those `_s??`"), but at the end of the list you can see the names of the built-in string methods. It is so easy to get this list that it soon becomes familiar and convenient. The documentation of a method can be accessed by printing out its doc-string like this:

```
>>> print "".isalpha.__doc__
S.isalpha() -> int
```

```
Return 1 if all characters in S are alphabetic
and there is at least one character in S, 0 otherwise.
```

Python does have more extensive documentation available on-line at www.python.org.

Another particularly useful Python documentation feature is encapsulated in the doctest module. This module lets you cut-and-paste text directly from a Python interactive shell session into a function doc-string, and then automatically re-run the commands in the session, checking that their output has remained the same. For instance, we can write the quadratic function like this:

```
def quadratic(a, b, c):
    """ Returns both solutions of the quadratic
    equation ax^2 + bx + c = 0.

    >>> quadratic(1, 2, 3)
    ((-1+1.4142135623730951j), (-1-1.4142135623730951j))
    >>> quadratic(1, 1, -2)
    ((1+0j), (-2+0j))
    """
    import cmath
    sol1 = (-b + cmath.sqrt(b ** 2 - 4*a*c)) / 2
    sol2 = (-b - cmath.sqrt(b ** 2 - 4*a*c)) / 2
    return sol1, sol2
```

The value is twofold. The added documentation shows a simple example of how to use the function, which can be immensely useful to programmers who want to use it. Plus, doctest can be run on the function, which automatically parses the doc-string, extracts the interpreter commands (they are preceded by `">>>"`), and then checking to see if the output using the current version of the function matches (as a string) the output in the documentation. This can be used for simple regressions testing.[\[note 7\]](#)

5. The New World of Programming

Having seen some of the features that make Python such an appealing language, I now want to present an argument for using Python as a first programming language. Any discussion of programming languages is in danger of degenerating into a religious battle. Most of the arguments and evidence is anecdotal, based on the particular experiences and tastes of practicing educators. This is good and bad. Much of the advice that one finds for teaching programming is eminently practical, tried and tested in the classroom. However, it can also be a narrow point of view, tied to a kind of programming that does not match the advances in programming styles and approaches commonly used in industry. Another difficulty is the fact that computer programming is a relatively young field, and what constitutes best practice is still being learned. The rise of "object's first" can be seen as an effort to bring the full benefits of objects into the classroom. Objects and classes have earned their way into the canon of basic programming constructs, alongside loops, lists, and if-statements. We are in the middle of a re-organization of the basic concepts of programming.

Yet there is a larger re-shaping of society that we are also part of: the World Wide Web has become a major element of modern society, as influential as the phone, automobile, or television. The average students who come out of high school have typically been using computers and the web for *years*, and aiming for a career as a web developer/designer now seems respectable. Anyone who uses a computer to do any significant authoring, whether it be for a website, a spreadsheet, a database, or a traditional programming language, is essentially engaged in a form of programming. Thus, ordinary computer users can become better users and consumers of computer technology by learning how to program, in the same way that the average car user will be a better driver by learning more about how their cars engine works.

This is not the traditional computer science or engineering perspective. The implicit assumption in traditional computer science is that students are aiming to become software engineers, or want to be involved on a daily basis with the construction of large software programs such as operating systems, databases, or compilers. These are the bread and butter tools of all programming, and are certainly things that software engineers need to know about, but in the new world of programming not all programmers want to be, or in fact need to be, software engineers. Everyone does programming, in the same sense that everyone drives a car. Every day, web developers write mark-up, or a few lines of scripting code (in addition to doing many other things that do not look anything like programming or computer science). System administrators need to write scripts and master the dozens of options for installing and configuring networks and software. Business people use spreadsheets that look very much like a graphical form of programming. Databases and web search engines have query languages that, even if they are not full-blown computation schemes, can be seen as a limited form of programming. Many complex application packages even come with their own scripting languages.

If we agree that essentially all students should learn to program, then the question is what to teach them in their first programming course. Naturally, students who have already decided their course of study may prefer to learn a particular language, e.g. engineering students might want Matlab and C, while arts students might prefer scripting languages found in popular multimedia packages. Yet for most students, a general-purpose introduction to programming that does not depend solely on the product-of-the-day is preferable. The problem with computer science introductions to programming is that they usually teach languages like Java or C++, which are not general-purpose languages. Certainly, they are called general-purpose high-level languages, but that is only true in the context of software engineering. They are not general-purpose from the point of web designers, or people who do programming but not as the main portion of their job. What it means to do programming, and to be a programmer, is changing before our very eyes.

While scripting languages have only a marginally better reputation than BASIC in computer science circles, that reputation is undeserved for Python. As mentioned earlier, Python has been likened to a practical version of Scheme, and the comparison is accurate; the major difference is the syntax and availability of better code libraries in Python. Many of the advantages for teaching Scheme also apply to Python.^[14] Python has the additional benefits of having a large, vibrant community of users, many of whom are professional programmers using Python daily. The problem with teaching Scheme to everyone is that, despite all its elegance, the average student who goes on to do some sort of programming will probably never see anything like it again. The LISP family of languages (of which Scheme is a favoured son) is practically moribund in the real world of programming, and this can be discouraging to students who are bursting to write programs like the ones they have grown up using. The presence of a community is often ignored or underestimated when evaluating programming languages, at least within academia.

All of this, plus the reasons mentioned earlier, makes Python an excellent choice as a first programming language for a wide range of students. It's easy to learn, it's eminently practical, and it is infused with a generous helping of the concepts and techniques that computer scientists love. What more can you ask for in a programming language?

6. Python in CS1

While one might believe that students in general would benefit from learning Python, it is not so obvious that traditional computer scientists would be best served by it. Besides "I've never heard of Python", some of the common arguments against Python as a first language are:

- It's a scripting language.
- It doesn't use C++/Java syntax.
- It lacks static type checking.
- There are comparatively few textbooks for learning Python.

The first two objections can be answered easily: complaining that Python is a scripting language is merely name-calling, and not an argument. Typically, this issue is raised by people who have had poor experiences with other scripting languages such as Perl or TCL/Tk, and they automatically transfer the same criticism to Python. But experience shows that Python is different: it is better designed and easier to read than most other scripting languages. The fact that it does not use C++/Java syntax is really only a problem if you like C++/Java syntax, or put an extraordinarily high value on the use of the particular tokens and keywords these languages happen to use. Python is structurally quite similar to C++/Java. It has expressions, statements, strings, functions, and classes that are expressed roughly in the same way as in C++/Java, albeit sometimes with different tokens and conventions. Another common critique, mainly from people who have never used Python, or who have only briefly dabbled in it, is that the way it uses whitespace to represent code blocks is error-prone and unethical. However, experience shows that this it is rarely a problem, and in fact it tends to be an extremely positive feature that promotes source code readability.

The issue of static type checking is a much larger debate, and there are many arguments for it, pro and con. For instance, on the one hand, static type checking is good because it lets the compiler catch errors early in the development process. On the other hand, static type checking makes certain techniques quite difficult to use, and require much more up-front design on the part of programmers. Up-front design is no doubt a good thing, but as the name "static" suggests, you cannot change the type of a static object. If you are solving a problem in a restricted, well-defined domain, such as mathematics, or the insular world of traditional first-year programming puzzles, then static typing is clearly a win since the problems are usually completely and consistently defined. In reality, many programming problems lack that sort of definitional rigor, or the requirements change midway through development. The ability to easily refactor and modify an existing piece of code is extremely useful and important, and a dynamically-typed language makes this much easier. For this reason, dynamically typed languages also tend to be better choices for prototyping. What is clear is that computer science students should be introduced to many different styles of languages throughout their education, and that should include a dynamically typed language.

Finally, the last argument on this list is a pragmatic and important one with respect to the adoption of Python in any serious way for large courses: there are indeed relatively few good textbooks for learning Python. But of course, you only need one textbook for a programming course, and quality is more important than quantity. There are some. Deitel & Deitel have published a heavy Python book, and as is usual, the Deitel's thorough but dry style seems to polarize readers.^[15] The book Learning Python could potentially be used as an introductory textbook, but it would likely require more supplemental exercises and examples. There is a free, open source textbook appropriate for high school students called "How to Think Like a Computer Scientist". This has the unique advantage of making its text freely available to anyone who wants to *modify* it, and so the text could be tailored by individual instructors.^[16]

Notes

1. It's worth noting that Zope Corporation is now where the core Python development team is employed. Zope is an open source website management system for dynamic websites, and it is the premier example of a large application written mostly in Python.
2. Many sophisticated IDEs are available for Java. For instance: JCreator (<http://jcreator.com/>), Netbeans (<http://www.netbeans.org/>), Eclipse (<http://www.eclipse.org/>), or IntelliJ IDEA (<http://www.intelliij.com/idea>).
3. Future versions of Java may fix this problem through the addition of auto-boxing of primitive types.
4. The expression " $a < b < c$ " happens to be legal C++, but it doesn't evaluate the way you would expect. First " $a < b$ " is evaluated as either true or false, and the result is cast to 0 if false, and some non-zero value if true. For instance, if " $a < b$ " is false, then the original expression amounts to evaluation " $0 < c$ ".
5. This problem comes from p.61 of Edsger Dijkstra's A Discipline of Computer Programming (1976, Prentice Hall). Python shares many of the nice features of the theoretical programming language that Dijkstra defines, and the Python solution is close to the one given there.
6. For those familiar with functional programming, list comprehensions are a notation that combines, and largely obviates the need for, the traditional map and filter functions.
7. For more rigorous testing, Python comes with PyUnit, a Python version of the popular JUnit unit testing framework.

References

1. [The Python website, http://www.python.org](http://www.python.org)

2. Tanenbaum, A.S., Renesse, R. van, Staveren, H. van., Sharp, G.J., Mullender, S.J., Jansen, A.J., and Rossum, G. van: "Experiences with the Amoeba Distributed Operating System," Communications of the ACM, vol. 33, pp. 46-63, Dec. 1990. (also on-line at <http://www.cs.vu.nl/pub/amoeba/>)
3. The ABC Language, <http://www.cwi.nl/~steven/abc/>
4. The Python Project on SourceForge.net, <http://sourceforge.net/projects/python/>
5. Python PEPs, <http://www.python.org/peps/>
6. Rossum, G. van: "Computer Programming for Everyone", revised DARPA funding proposal, August 1999, <http://www.python.org/doc/essays/cp4e.html>
Rossum, G. van: "Computer Programming for Everyone", revised DARPA funding proposal, August 1999, <http://www.python.org/doc/essays/cp4e.html>
7. Martelli, A., Ascher D. (eds): "The Python Cookbook", O'Reilly, 2003
8. Raymond, E.: "Why Python", The Linux Journal, Issue 73, May 2000, <http://www.linuxjournal.com/article.php?sid=3882>
9. Peter Norvig's website, <http://www.norvig.com>
10. Python success stories, Python Business Forum, <http://www.python-in-business.org/success/>
11. Norvig, P.: Python for Lisp Programmers, essay at <http://www.norvig.com/python-lisp.html>
12. Stackless Python, A Python Implementation that Does Not Use the C Stack, <http://www.stackless.com/>
13. BlueJ website, <http://www.bluej.org/>
14. Felleisen, M., Findler, B.R., Flatt, M., Krishnamurthi, S., How to Design Programs, MIT Press, Sept. 2002
<http://www.htdp.org/>
15. Harvey M. Deitel, H. M. (Editor), Deitel P. J., Liperi J. P., Wiedermann B. A., P. Liperi, J. P., "Python, How to Program", Prentice Hall, Feb. 2002
16. Downey, A.B., Elkner, J., Meyers, C., "How to Think Like a Computer Scientist, Learning with Python", an open source book on-line at <http://www.ibiblio.org/obp/thinkCSpy/>